

JavaScript-16

Introduktion till Vue

30 september 2026

JavaScript – utan tjafs
Författare: Acke Strömberg



För en kanin

Introduktion till Vue JS

I de föregående kapitlen har vi djupdykt i React och sett hur det hanterar gränssnitt genom att blanda HTML och JavaScript i JSX-filer. React använder ett så kallat enkelriktat dataflöde, vilket innebär att vi manuellt måste tala om för motorn när ett värde ska ändras via en set-funktion (setAntal).

Vue JS har blivit känt i branschen för att ha en betydligt skonsammare inlärningskurva. Vue delar inte bara upp koden i traditionella block (HTML, JS och CSS separat), utan introducerar också Tvåvägs-databindning (Two-way data binding). I det här kapitlet ska vi titta närmare på hur detta fungerar under huven med hjälp av Vues moderna Composition API och smarta beräknade värden (Computed Properties).

Composition API och Reaktivitet med ref()

I moderna Vue-applikationer (Vue 3 och framåt) använder vi ett upplägg som skrivs via `<script setup>`. Det gör att vi slipper krångliga inställningar och kan skriva ren, minimalistisk JavaScript-kod. För att skapa en variabel som komponenten ska komma ihåg, packar vi in den i en inbyggd Vue-funktion som heter `ref()`. Det talar om för Vue att variabeln ska vara reaktiv – så fort värdet ändras i minnet ska skärmen uppdateras automatiskt.

Det dolda objektet under huven

När du skapar en variabel med `ref()`, gör Vue om din variabel till ett dolt objekt. Detta skapar en viktig JavaScript-regel som alla nybörjare snubblar på:

- Inuti din HTML-mall (`<template>`) skriver du bara variabelns namn precis som vanligt.
- Men inuti ditt JavaScript-block (`<script>`) måste du skriva `.value` efter namnet för att kunna läsa eller ändra på värdet!

```
<script setup>
import { ref } from 'vue';

// Vi skapar ett reaktivt minne
const meddelande = ref('Hej klassen!');

function uppdateraText() {
  // VIKTIGT: Inuti script-blocket MÅSTE vi använda .value!
  meddelande.value = 'Hoppsan, texten ändrades!';
}
</script>

<template>
  <div>
    <!-- I HTML-mallen behövs INTE .value, Vue packar upp det automatiskt -->
    <h1>{{ meddelande }}</h1>
    <button @click="uppdateraText">Ändra texten</button>
  </div>
</template>
```

Bild 1 Vue använder dolda objekt-proxies under huven för att känna av exakt när du ändrar på `.value` och ritar då direkt om gränssnittet.

Strukturen i en Single File Component (.vue)

Medan React blandar allt i `.jsx`-filer, håller Vue kvar vid det traditionella webbtänket där vi separerar kodspråken, fast inuti en och samma fil. En `.vue`-fil kallas för en Single File Component (SFC) och är snyggt uppdelad i tre tydliga block.

1. `<script setup>`: Här bor din JavaScript-kod, dina importers och dina reaktiva variabler.
2. `<template>`: Här bor din HTML-struktur. Det är detta som faktiskt ritas ut på skärmen.
3. `<style scoped>`: Här bor din CSS-styling. Nyckelordet `scoped` är en superkraft som gör att stylingen bara gäller för just den här komponenten och kan aldrig råka spilla över och förstöra designen på andra sidor.

Tvåvägs-databindning (v-model)

Det absolut mest populära verktyget i Vue kallas för v-model. Det används för att koppla ihop ett input-fält (som ett sökfält eller ett formulär) med en reaktiv variabel i din kod. Det fantastiska med v-model är att kopplingen är tvåvägs:

- När användaren skriver i textfältet uppdateras din JavaScript-variabel automatiskt.
- Om du ändrar på variabeln inifrån din JavaScript-kod uppdateras textfältet på skärmen automatiskt.

Jämförelse (React vs Vue):

Titta på hur mycket mindre kod du behöver skriva i Vue jämfört med React för att synka ett textfält med en variabel:

```
// SÅ HÄR GÖR MAN I REACT (Enkelriktat - mycket kod):  
<input type="text" value={namn} onChange={(e) => setNamn(e.target.value)} />
```

```
<!-- SÅ HÄR GÖR MAN I VUE (Tvåvägs - "magiskt" och kort): -->  
<input type="text" v-model="namn" />
```

Bild 2 Med v-model slipper du skriva manuella händelselyssnare (onChange/input) för att hämta text från formulär – Vue sköter allt synkande i bakgrunden.

Vues Directives (Smarta HTML-attribut)

I React använder vi vanlig JavaScript (.map(), if-satser) inuti vår JSX för att styra vad som visas. Vue har valt en annan väg. De använder något som kallas Directives. Det är speciella attribut som man sätter direkt på HTML-taggar (alla startar med v-). Här är de tre viktigaste som du måste kunna.

- v-on (eller @) – Händelselyssnare
Istället för onClick använder Vue v-on:click, vilket nästan alltid förkortas till bara @click.

```
<button @click="antal++">Klicka här</button>
```

- v-if – Visa om ett villkor är sant
Om du vill att ett element bara ska synas om ett visst villkor är uppfyllt använder du v-if.

```
<h2 v-if="arInloggad">Välkommen tillbaka!</h2>  
<h2 v-else>Vänligen logga in...</h2>
```

- v-for – Loopa ut listor

Istället för `.map()` har Vue en inbyggd loop-sats direkt på HTML-taggen som heter `v-for`. Den fungerar nästan exakt som en vanlig `for...of`-loop i JavaScript.

```
<ul>
  <!-- Vi loopar ut frukterna direkt på li-taggen -->
  <li v-for="(frukt, index) in frukter" :key="index">
    {{ frukt }}
  </li>
</ul>
```

Computed Properties – Smarta beräknade värden

När vi bygger större gränssnitt vill vi ofta göra beräkningar på vår data (t.ex. räkna ut totalsumman för en kundvagn, eller filtrera en lista baserat på vad användaren har skrivit i ett sökfält). I React görs dessa beräkningar om från början vid varje rendering, vilket kan bli långsamt om man har mycket data. Vue löser detta med Computed Properties via funktionen `computed()`.

Ett beräknat värde (`computed`) är en smart funktion som minneslagrar (cachar) sitt resultat. Den körs bara om en gång om den data som funktionen beror på faktiskt ändras. Om datan är densamma, hämtar Vue blixtsnabbt det färdiga resultatet direkt ur cache-minnet!

Här knyter vi ihop alla delar vi har lärt oss: SFC-strukturen, tvåvägs-databindning, directives och computed properties.

```
<script setup>
import { ref, computed } from 'vue';

const sökord = ref('');
const produkter = ref(['Smartphone', 'Hörlurar', 'Smartwatch', 'Laptop']);

// En computed property och smart funktion som filtrerar listan i realtid
const filtreradeProdukter = computed(() => {
  return produkter.value.filter(produkt =>
    produkt.toLowerCase().includes(sökord.value.toLowerCase())
  );
});
</script>
```

```
<template>
  <div class="sök-box">
    <!-- Tvåvägs-databindning synkar sökordet direkt -->
    <input type="text" v-model="sökord" placeholder="Sök produkt..." />

    <h3>Resultat:</h3>
    <ul>
      <!-- Vi loopar ut det smarta, beräknade värdet istället för original-
      listan -->
      <li v-for="(produkt, index) in filtreradeProdukter" :key="index">
        {{ produkt }}
      </li>
    </ul>
  </div>
</template>

<style scoped>
.sök-box { padding: 20px; background: #f8f9fa; border-radius: 8px; }
input { padding: 8px; width: 100%; border: 1px solid #ccc; }
ul { list-style: none; padding: 0; }
li { padding: 8px 0; border-bottom: 1px solid #eee; }
```

Bild 3 computed() är ett ovärderligt verktyg för prestanda. Det ser till att tunga filtreringar eller uträkningar inte belastar webbläsaren i onödan.